

for Loop in Java

The `for` loop is a control flow statement that allows code to be executed repeatedly based on a condition. It is generally used when the number of iterations is known ahead of time. The loop consists of three parts: initialization, condition, and iteration.

Syntax:

```
for (initialization; condition; iteration) {  
    // Code to be executed  
}
```

- **initialization**: This step is executed once, and it is used to initialize a loop variable.
 - **condition**: This boolean expression is evaluated before each iteration. If it evaluates to `true`, the loop executes. If it evaluates to `false`, the loop terminates.
 - **iteration**: This statement is executed after each iteration and is usually used to update the loop variable (e.g., increment or decrement).
-

How It Works:

1. The initialization part is executed once, before the loop starts.
 2. The condition is evaluated. If it is `true`, the code inside the loop is executed.
 3. After executing the code block, the iteration part is executed (typically incrementing or decrementing the loop variable).
 4. The condition is checked again, and if it is still `true`, the loop executes again.
 5. This continues until the condition evaluates to `false`.
-

Example 1: Basic `for` Loop

```
public class Main {  
    public static void main(String[] args) {  
        for (int count = 1; count <= 5; count++) {  
            System.out.println("Count: " + count);  
        }  
    }  
}
```

Explanation:

- The loop starts with `count = 1`.
- The condition `count <= 5` is checked, and if it's `true`, the code inside the loop is executed.
- After each iteration, `count` is incremented by 1 (`count++`).
- The loop continues until `count` becomes `6`, at which point the condition becomes `false`, and the loop terminates.

Output:

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

Example 2: Loop with Decrement

```
public class Main {
    public static void main(String[] args) {
        for (int count = 5; count >= 1; count--) {
            System.out.println("Count: " + count);
        }
    }
}
```

Explanation:

- The loop starts with `count = 5`.
- The condition `count >= 1` is checked, and if it's `true`, the code inside the loop is executed.
- After each iteration, `count` is decremented by 1 (`count--`).
- The loop continues until `count` becomes `0`, at which point the condition becomes `false`, and the loop terminates.

Output:

```
Count: 5
Count: 4
Count: 3
```

Count: 2

Count: 1

Example 3: `for` Loop with Multiple Variables

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 1, j = 10; i <= 5; i++, j--) {  
            System.out.println("i: " + i + ", j: " + j);  
        }  
    }  
}
```

Explanation:

- The loop starts with two variables: `i = 1` and `j = 10`.
- The condition `i <= 5` is checked.
- After each iteration, `i` is incremented by 1 (`i++`), and `j` is decremented by 1 (`j--`).
- The loop continues until `i` becomes `6`, at which point the condition becomes `false`, and the loop terminates.

Output:

```
i: 1, j: 10  
i: 2, j: 9  
i: 3, j: 8  
i: 4, j: 7  
i: 5, j: 6
```

Example 4: Nested `for` Loop

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 2; j++) {  
                System.out.println("i: " + i + ", j: " + j);  
            }  
        }  
    }  
}
```

```
}  
}
```

Explanation:

- The outer loop runs 3 times (`i` goes from 1 to 3).
- For each iteration of the outer loop, the inner loop runs 2 times (`j` goes from 1 to 2).
- The inner loop executes completely for each iteration of the outer loop.

Output:

```
i: 1, j: 1  
i: 1, j: 2  
i: 2, j: 1  
i: 2, j: 2  
i: 3, j: 1  
i: 3, j: 2
```

Example 5: `for` Loop with `break` Statement

```
public class Main {  
    public static void main(String[] args) {  
        for (int count = 1; count <= 5; count++) {  
            if (count == 3) {  
                break; // Exit the loop when count equals 3  
            }  
            System.out.println("Count: " + count);  
        }  
    }  
}
```

Explanation:

- The loop prints the numbers from 1 to 2.
- When `count` equals 3, the `break` statement is executed, which immediately exits the loop.
- The loop does not print 3 and stops after printing 2.

Output:

```
Count: 1
```

Count: 2

Example 6: for Loop with continue Statement

```
public class Main {  
    public static void main(String[] args) {  
        for (int count = 1; count <= 5; count++) {  
            if (count == 3) {  
                continue; // Skip this iteration when count equals 3  
            }  
            System.out.println("Count: " + count);  
        }  
    }  
}
```

Explanation:

- The loop will print numbers from 1 to 5, but when `count` equals 3, the `continue` statement is executed.
- The `continue` statement skips the current iteration (does not execute `System.out.println` for `count = 3`), and the loop proceeds with the next iteration.

Output:

```
Count: 1  
Count: 2  
Count: 4  
Count: 5
```

Key Points:

1. **Initialization:** The loop variable is initialized before the loop starts and typically only once.
2. **Condition:** The condition is checked before each iteration. If it evaluates to `false`, the loop terminates.
3. **Iteration:** The loop variable is updated (incremented or decremented) after each iteration.
4. **Break Statement:** Used to exit the loop prematurely when a certain condition is met.

5. **Continue Statement:** Skips the current iteration of the loop and proceeds with the next iteration.
-